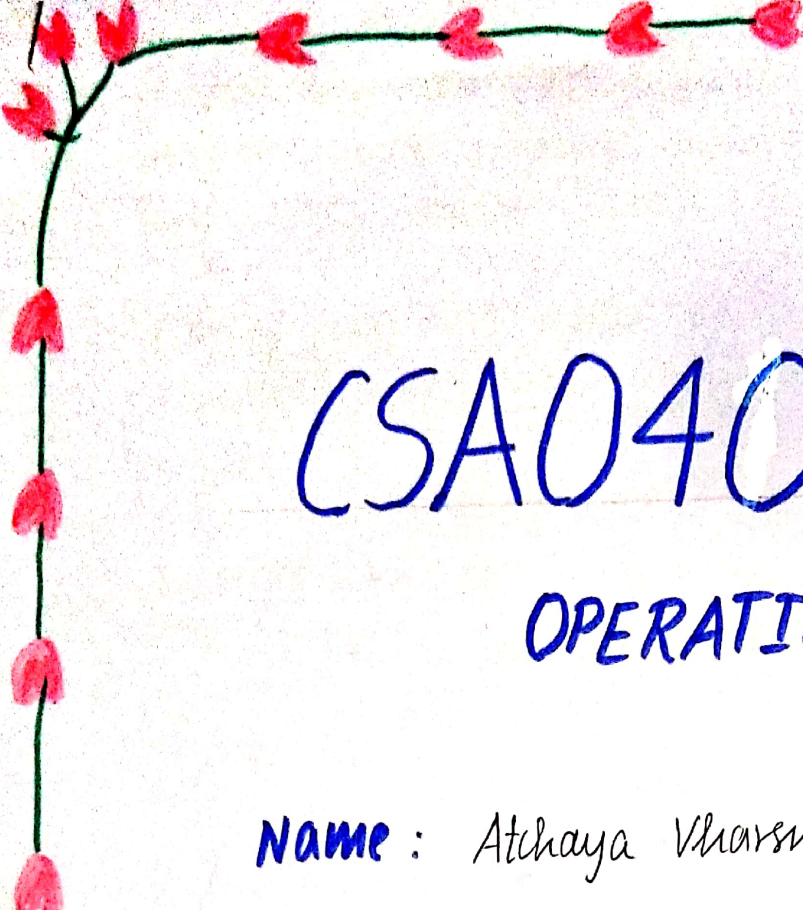




CSA0401

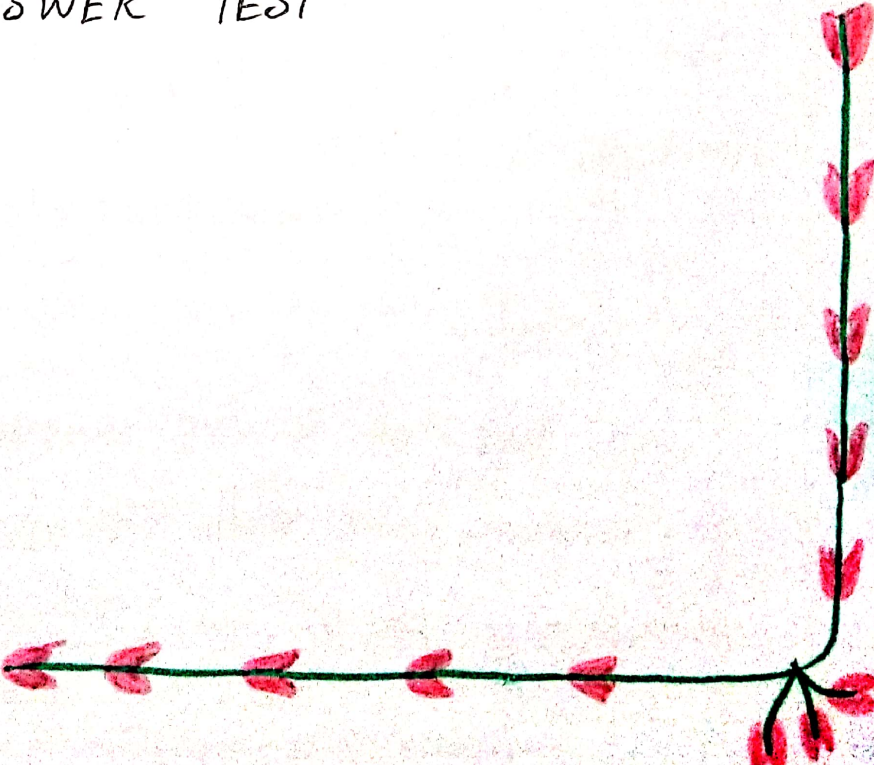
OPERATING SYSTEMS

Name : Atchaya Vharsne S

Reg. No. : 192524185

CO1 AT1

SHORT ANSWER TEST



1. Suppose a modern OS supports multiple users and processes with shared resources like CPU, memory and I/O devices.

a) Classify different types of OS suitable for this scenario and justify your choice.

TYPES OF OPERATING SYSTEMS :

1. Multi-user OS :

- Allows multiple users to use the computer at the same time.
- Ex: Linux, UNIX.

2. Multitasking OS :

- Runs multiple processes simultaneously.
- Several apps can execute together.

3. Multiprocessing OS :

- Uses more than one CPU/core
- Improves speed and executes many processes efficiently.

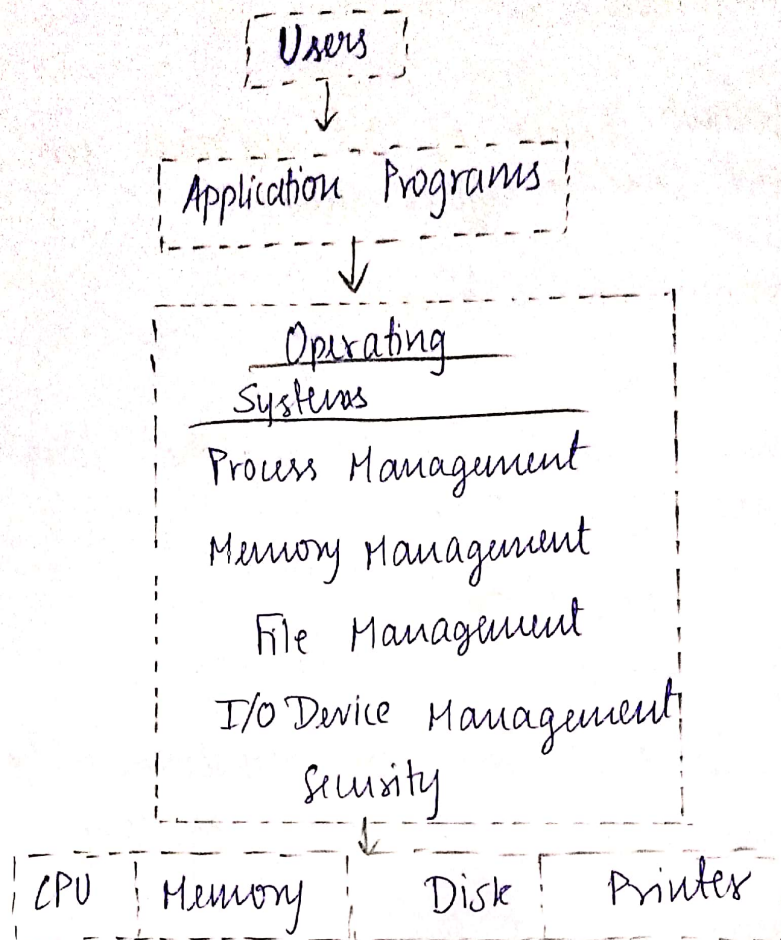
4. Time sharing OS :

- Gives each process a small amount of CPU time.
- Every user gets a quick response.

SUITABLE CHOICE :

A multi-user Time-sharing OS is the best choice because it allows multiple users and processes to share CPU, memory & I/O devices efficiently.

b> Draw the structure of an OS showing how resources are managed.



Explanation :

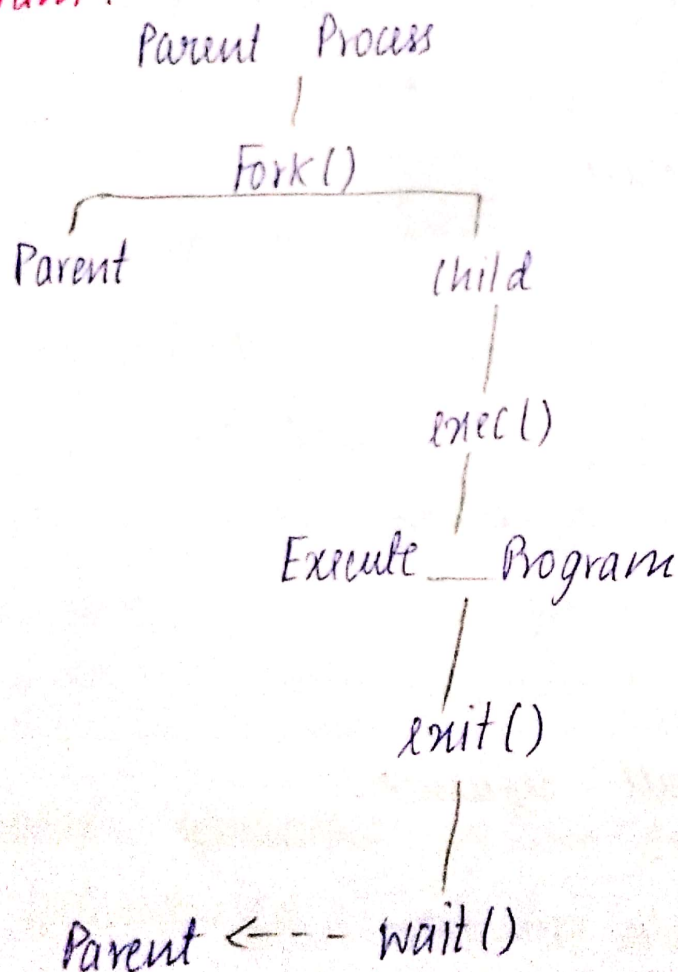
- Users run Applications.
- Applications request services from the OS.
- The OS manages CPU, memory, files & I/O Devices.
- Hardware resources are shared efficiently among all users & processes.

c) write a sequence of system calls required to create and execute a new process in such an environment.

Sequence of system calls.

1. Fork() - creates a new child process.
2. exec() - loads & executes a new program in the child process.
3. wait() - Parent process waits until the child finishes.
4. exit() - child process terminates after execution.

Flow Diagram:



2. Consider an OS managing multiple files and devices for concurrent users.

a) categorize system calls used for file & device management

File management

- `open()` - opens a file
- `read()` - Reads data
- `write()` - writes data
- `close()` - closes a file

Device management

- `ioctl()` - controls a device
- `read()` - reads from a device
- `write()` - writes to a device.

b) Layered Structure.



c) C System call sequence.

```
int fd;  
char buf[100];  
fd = open("test.txt", O_RDONLY);  
read(fd, buf, 100);  
close(fd);
```


3. Assume a real-time Operating System used in an embedded system controlling industrial machinery.

a) Identify the type of OS & explain why it is suitable for real-time applications.

Real-Time OS

- Fast response
- Meets deadlines
- Used in industrial machines & embedded systems.

b) Draw a diagram representing the interaction b/w hardware, kernel, & user processes.

User Process

Kernel

Hardware.

c) Process Synchronization system calls.

- `fork()` - Create Process
- `wait()` - wait for another process
- `sem_wait()` - lock semaphore
- `sem_post()` - Release semaphore.

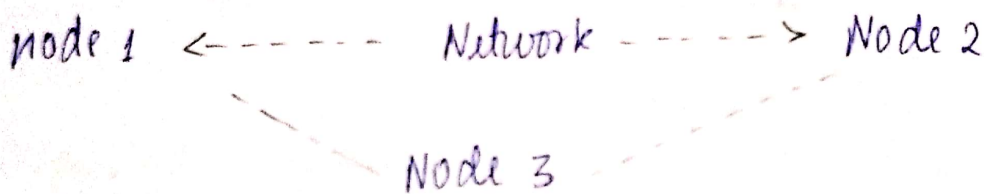
4. A distributed computing system consists of multiple interconnected computers sharing resources over a network.

a) classify the type of OS used and compare it with centralized OS
Distributed OS

Comparison

Distributed OS	Centralized OS
Many computers Shares resources	One computer local resources only.

b) Draw the architecture of a distributed OS showing communication b/w nodes.



c) IPC system calls.

- Pipe()
- fork()
- read()
- write()
- close()

5. Consider a scenario where an OS must manage process scheduling, memory allocation, and device communication efficiently.

a) Explain the major functions of an OS in handling these tasks.

- Process management - Schedules processes.
- Memory management - Allocates memory.
- Device Management - Controls I/O devices.

b) Draw a block diagram representing OS components and their interactions.

Users

Operating System

CPU Memory Devices.

c) Write system call examples for memory allocation and device management.

Memory

- `malloc()`
- `mmap()`

Device

- `open()`
- `read()`
- `write()`
- `close()`